

实验四实验报告

一、4 位先行进位加法器：

1.整体设计思路：

将 CLA 分为三个部分：

1) CLU：利用 FA 提供的 P_i 和 G_i 计算出每一位进位，其中 $P_i=A_i \cdot B_i$,

$$G_i=A_i+B_i$$

$$C_1 = G_0 + P_0C_0$$

$$C_2 = G_1 + P_1G_0 + P_1P_0C_0$$

$$C_3 = G_2 + P_2G_1 + P_2P_1G_0 + P_2P_1P_0C_0$$

$$C_4 = G_3 + P_3G_2 + P_3P_2G_1 + P_3P_2P_1G_0 + P_3P_2P_1P_0C_0$$

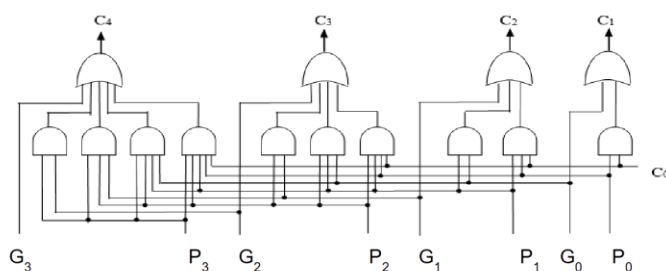
2) 支持进位生成函数和进位传递函数的全加器：

在全加器的基础上增加两个输出： $P_i=A_i \cdot B_i$ ， $G_i=A_i+B_i$ 便于更好的与 CLU 互动。

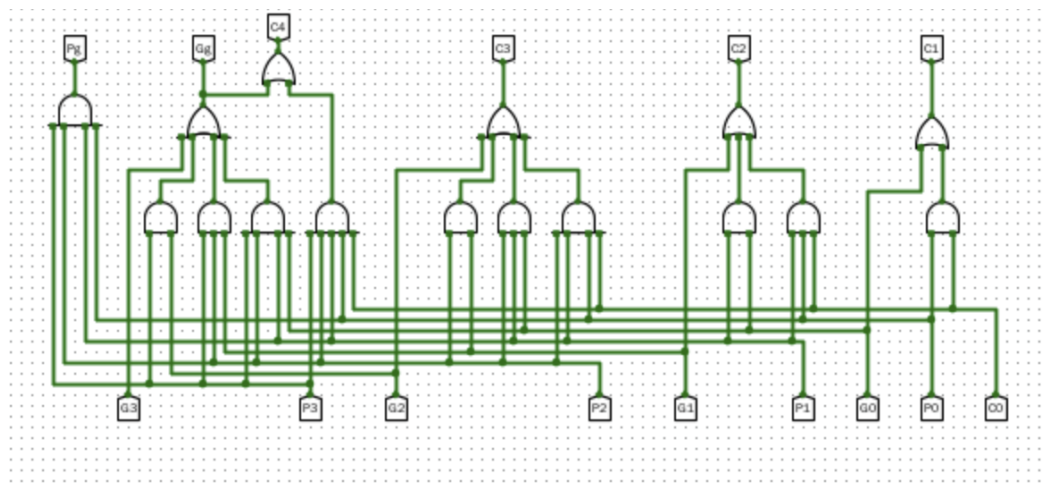
3) 主电路：将 CLU 与 FA 连接完成 CLA 的搭建。

2.子模块介绍与描述：

1) CLU 原理图：



CLU 电路图：

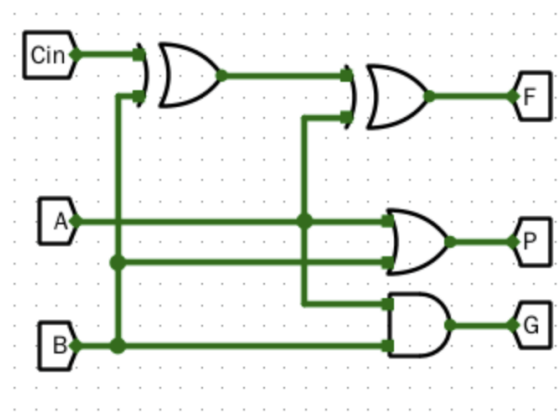


接口作用：

接口方向	接口名称	位宽	作用说明
输入	P0~P3	各 1 位	4 位全加器的进位传递函数输入
输入	G0~G3	各 1 位	4 位全加器的进位生成函数输入
输入	C0	1 位	最低位进位输入
输出	C1~C4	各 1 位	各位先行进位输出（C1~C3 组内进位，C4 组间进位）
输出	Pg	1 位	成组进位传递函数

接口方向	接口名称	位宽	作用说明
输出	Gg	1 位	成组进位生成函数

2) FA 电路图：

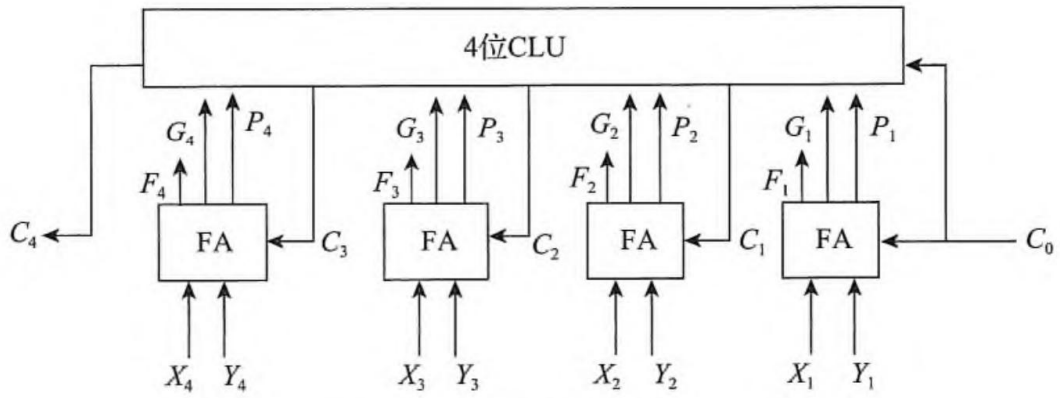


接口作用：

接口方向	接口名称	位宽	作用说明
输入	A	1 位	加法操作数 A
输入	B	1 位	加法操作数 B
输入	Cin	1 位	低位进位输入
输出	F	1 位	本位和 $(A \oplus B \oplus Cin)$
输出	P	1 位	进位传递函数 $(P=A+B)$

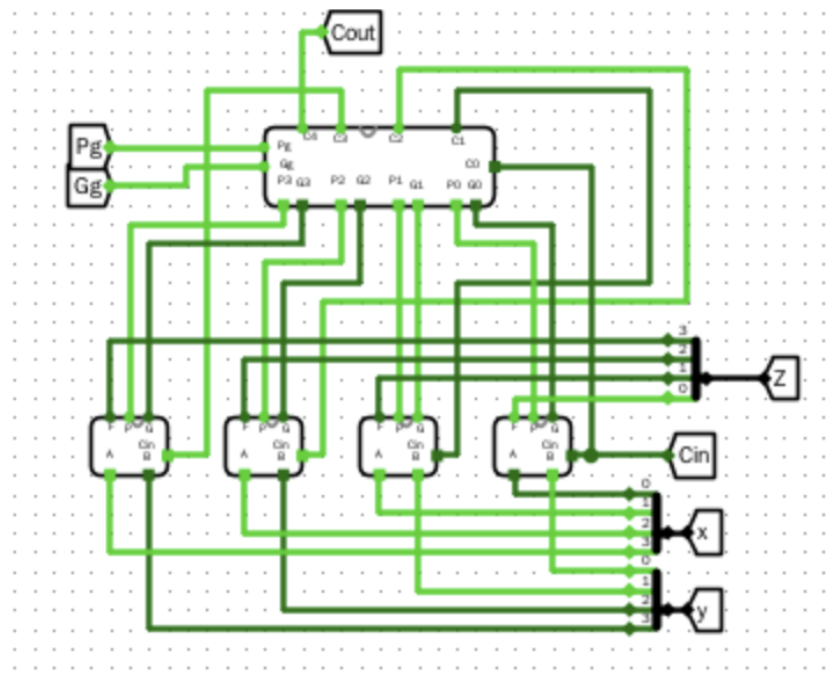
接口方向	接口名称	位宽	作用说明
输出	G	1 位	进位生成函数 ($G=A \cdot B$)

3) CLA 原理图:



b) 4位全先行进位加法器

CLA 电路图:



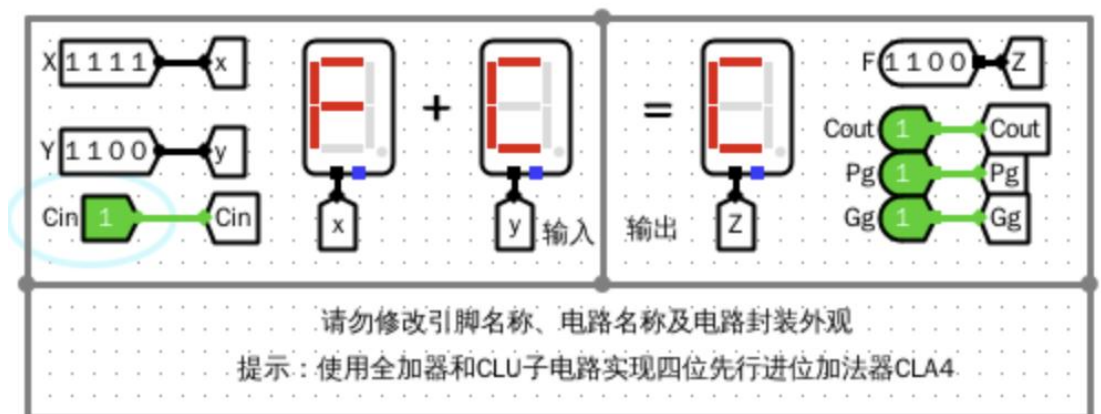
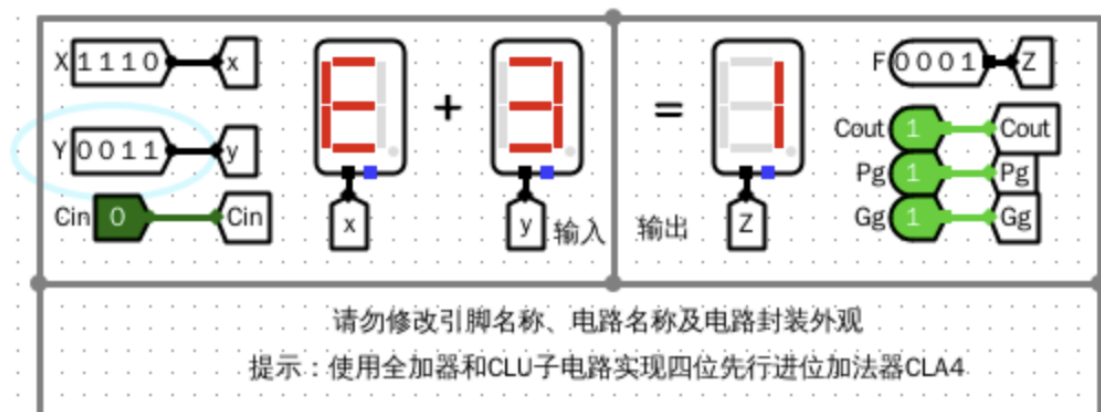
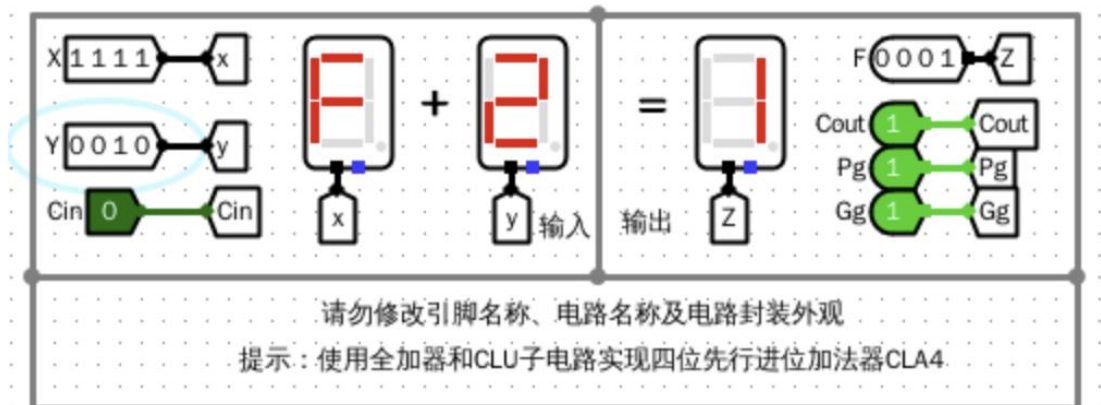
接口作用:

接口方向	接口名称	位宽	作用说明
输入	X[3:0]	4 位	第一组 4 位操作数
输入	Y[3:0]	4 位	第二组 4 位操作数
输入	Cin	1 位	整个 4 位加法器的低位进位
输出	F[3:0]	4 位	4 位加法和结果
输出	Cout	1 位	最高位进位输出（等于 C4）
输出	Pg	1 位	4 位组的成组进位传递函数
输出	Gg	1 位	4 位组的成组进位生成函数

3.功能表（4 位 CLA 的功能为完成 4 位加减法，这里列举部分数据）

Cnt	X	Y	Cin	Z	Cout	Gg	Pg
00	f	2	0	1	1	1	1
01	e	3	0	1	1	1	1
02	d	4	0	1	1	1	0
03	c	5	0	1	1	1	0
04	b	6	0	1	1	1	1
05	a	7	0	1	1	1	1
06	9	8	0	1	1	1	0
07	8	9	0	1	1	1	0

4.仿真模拟：



5. 错误现象及分析：

在完成本次实验的过程中，没有遇到任何错误。

二、16 位先行进位加法器：

1. 整体设计思路：类似 4 位先行进位加法器，在设计 16 位先行进位加法

器时，用 4 位先行进位加法器替换 FA，继续沿用原 CLU 部件，按照原理图组装成 16 位先行进位加法器。

2.子模块功能和描述：

- 1) 4 位先行进位加法器：原理图，电路图与接口功能完全同第一个实验。
- 2) CLU：复用第一个实验完成的部件，原理图、电路图、接口同上。
- 3) 主电路：

原理图：

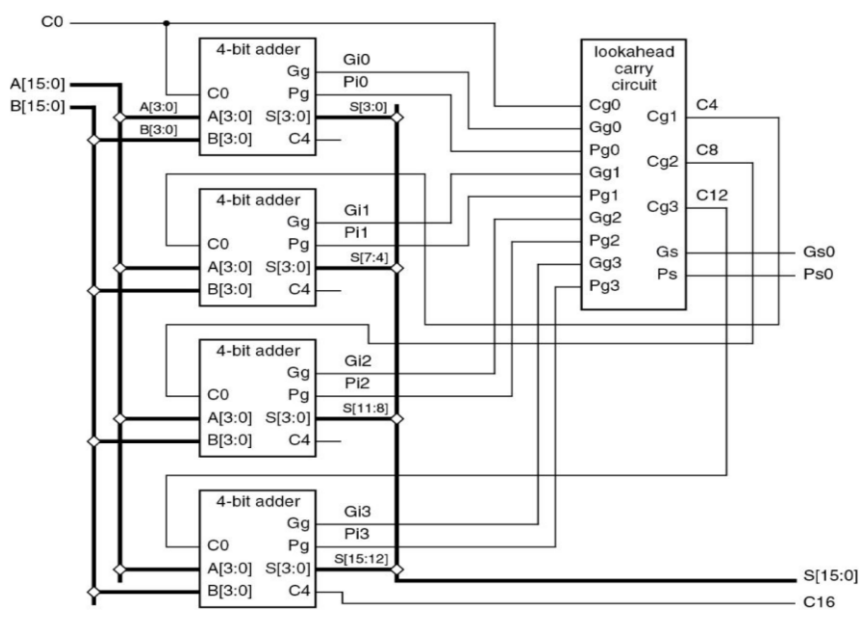
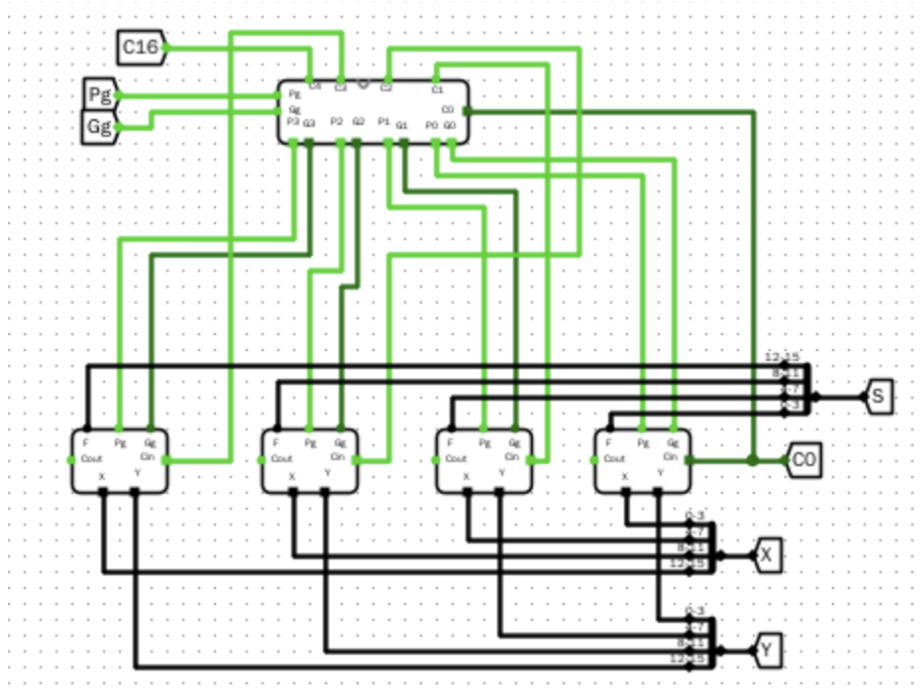


图 4.8 16 位两级先行进位加法器原理图

电路图：



接口描述：

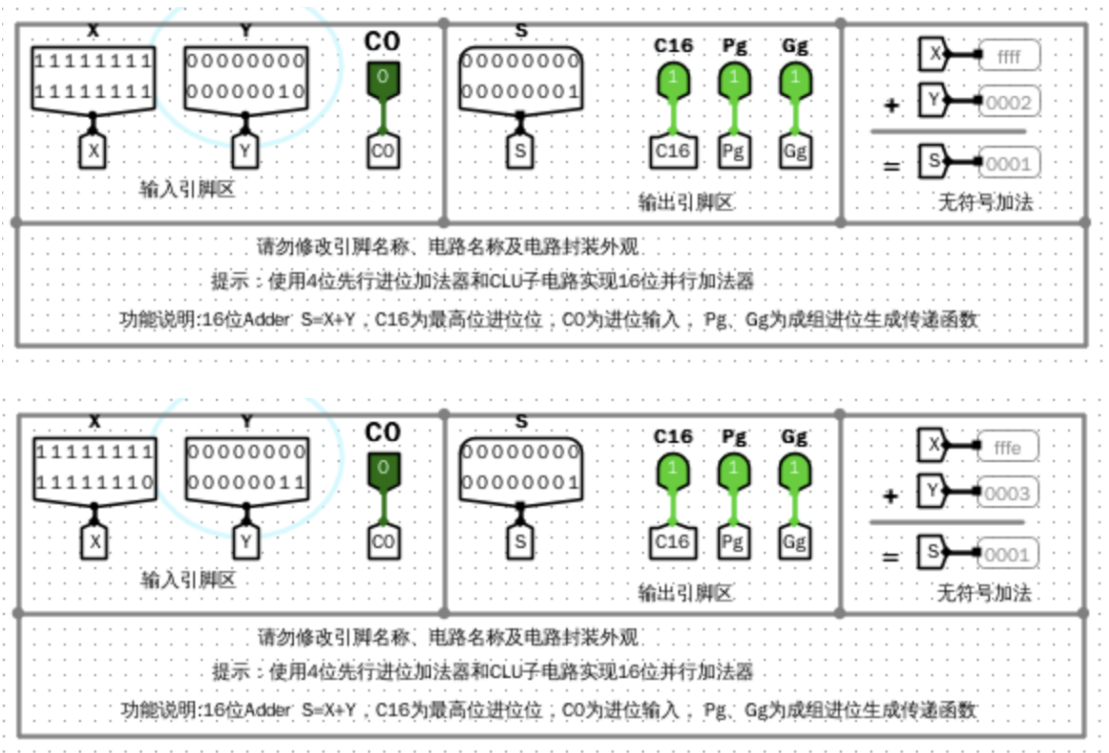
接口方向	接口名称	位宽	作用说明
输入	X[15:0]	16 位	16 位操作数 A（加数）
输入	Y[15:0]	16 位	16 位操作数 B（被加数）
输入	Cin	1 位	整个 16 位加法器的最低位进位输入
输出	S[15:0]	16 位	16 位加法和结果（ $S=X+Y$ ）
输出	C16	1 位	16 位最高位进位输出（最终进位）
输出	Pg	1 位	16 位整体成组进位传递函数

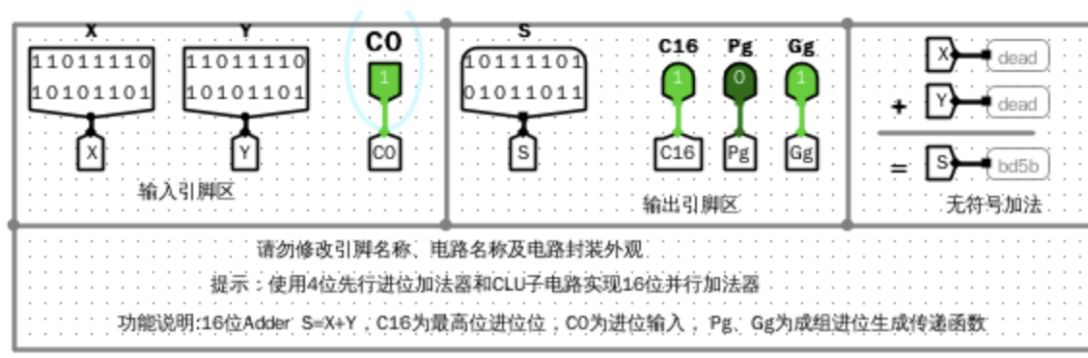
接口方向	接口名称	位宽	作用说明
输出	Gg	1 位	16 位整体成组进位生成函数

3.功能表（列举部分）

Cnt	X	Y	Cin	Z	Cout	Gg	Pg
00	ffff	0002	0	0001	1	1	1
01	fffe	0003	0	0001	1	1	1
02	fffd	0004	0	0001	1	1	0
03	fffc	0005	0	0001	1	1	0
04	fffb	0006	0	0001	1	1	1
05	fffa	0007	0	0001	1	1	1
06	fff9	0008	0	0001	1	1	0
07	fff8	0009	0	0001	1	1	0

4.仿真模拟：





5.错误现象及分析：

在完成本次实验的过程中，没有遇到任何错误。

三、32 位快速加法器：

1.整体模块设计：利用 4.2 设计好的 16 位先行进位加法器，用

$C_{16} = G_g + P_g \cdot C_{in}$ 算出第一个 16 位先行进位加法器的进位，直接传给第二个加法器，以此实现 32 位先行进位加法器。

同时利用

$$OF = A_{n-1} \cdot B_{n-1} \cdot F_{n-1} + A_{n-1} \cdot B_{n-1} \cdot F_{n-1};$$

$$SF = F_{n-1};$$

$$CF = Cout \oplus Cin;$$

$$ZF = \overline{F_{n-1} + F_{n-2} + \cdots + F_0} \quad (\text{实际设计中利用比较器完成})$$

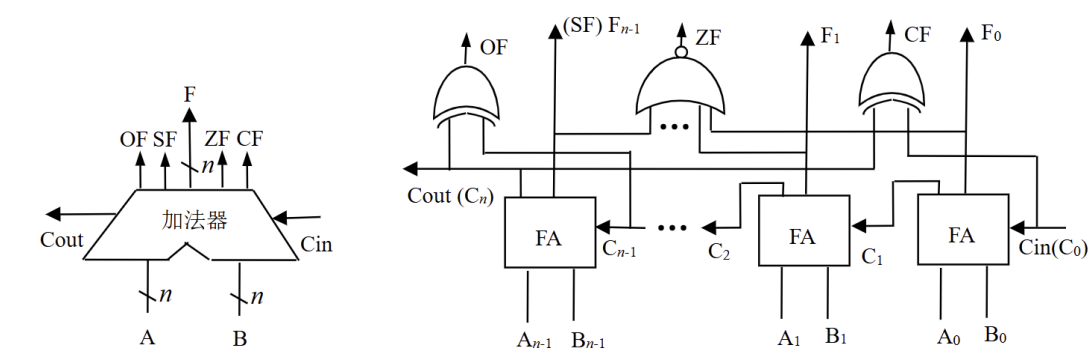
算出标志位。

2.子模块描述：

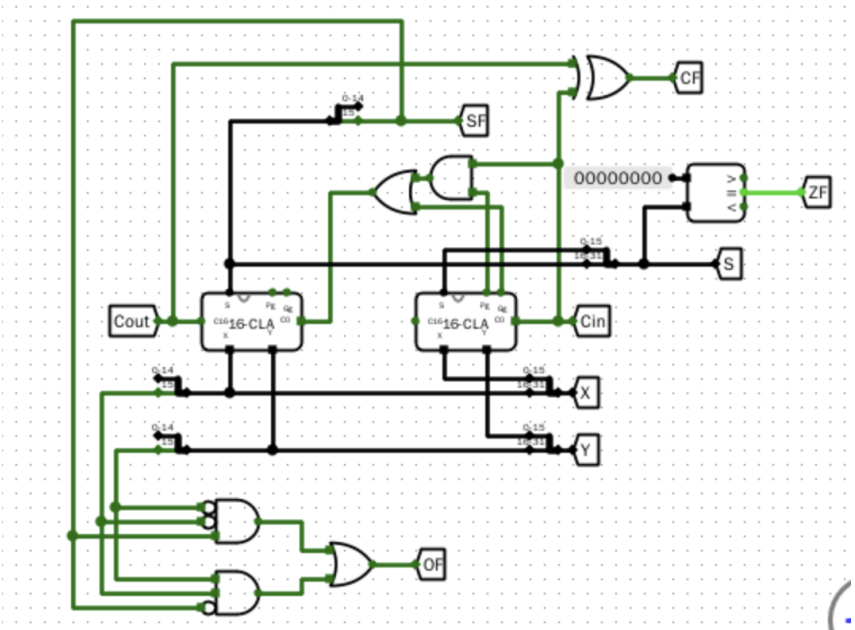
1) 16 位先行进位加法器：复用 4.2 的设计，原理图、电路图、接口等同上。

2) 主电路（32 位先行进位加法器）：

原理图：



电路图：



接口功能：

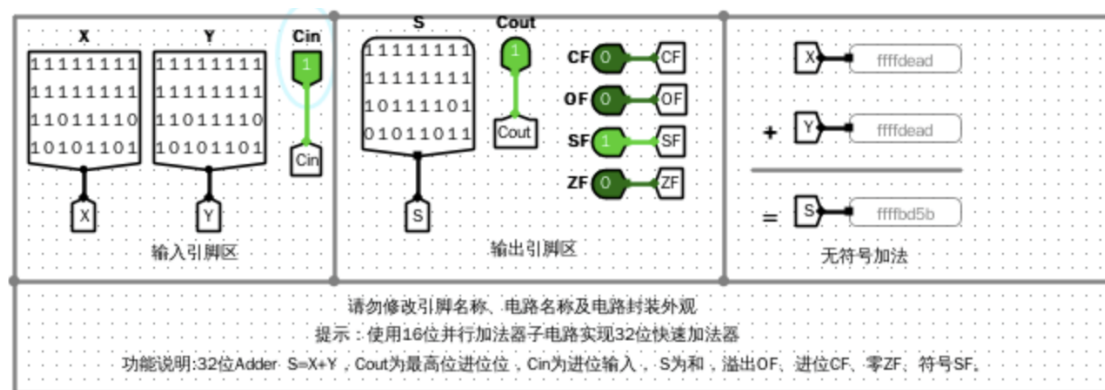
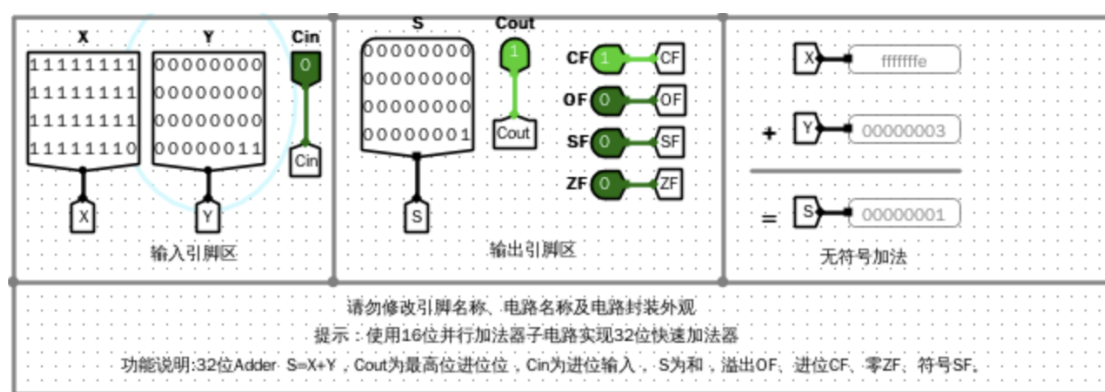
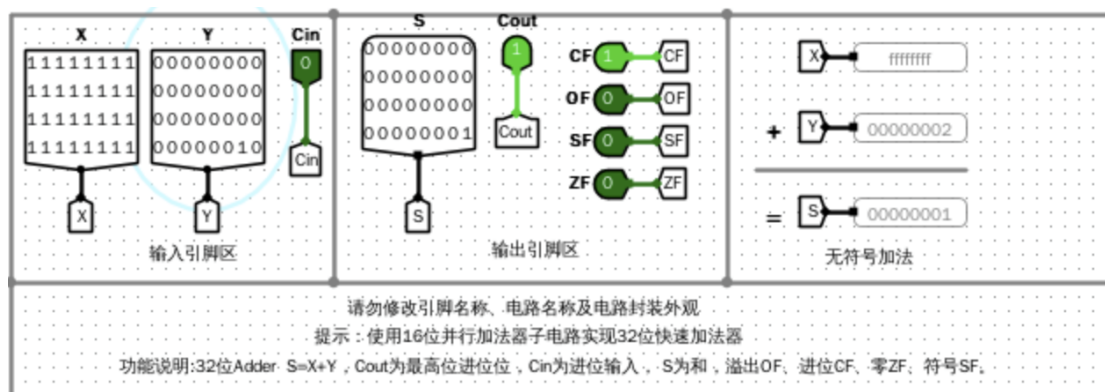
接口方向	接口名称	位宽	作用说明
输入	X[31:0]	32 位	32 位操作数 A

接口方向	接口名称	位宽	作用说明
输入	Y[31:0]	32 位	32 位操作数 B
输入	Cin	1 位	低位进位输入
输出	Result[31:0]	32 位	加法运算结果 (X+Y)
输出	Cout	1 位	最高位进位输出
输出	CF	1 位	进位标志 (无符号溢出)
输出	OF	1 位	溢出标志 (有符号溢出)
输出	SF	1 位	符号标志 (结果最高位)
输出	ZF	1 位	零标志 (结果全 0 为 1)

3.功能表（部分）

Cnt	X	Y	Cin	S	Cout	OF	CF	ZF	SF
00	ffffffff	00000002	0	00000001	1	0	1	0	0
01	fffffffef	80000003	0	80000001	1	0	1	0	1
02	7fffffffdf	00000004	0	80000001	0	1	0	0	1
03	fffffffefc	90000005	0	90000001	1	0	1	0	1
04	fffffffefb	00000006	0	00000001	1	0	1	0	0
05	fffffffefa	a0000007	0	a0000001	1	0	1	0	1
06	50000009	76543210	0	c6543219	0	1	0	0	1
07	708090ab	70000009	0	e08090b4	0	1	0	0	1

4.仿真模拟：



5.错误现象及分析：

在完成实验的过程中，没有遇到任何错误。

四、32 位桶形移位器：

1.整体模块设计：类比 8 位桶形移位器，增加两个多路选择器，（共五个，分别对应移位 1, 2, 4, 8, 16 位，可以线性组合为 0-31 的任意数字），每一个多路选择器利用 shamt 控制是否移位，L/R 控制移位方向，

(2 位二进制数共四个刚好对应所有移位情况)，A/L 配合一个 2 选一选择器控制移位方式，以此完成 32 位桶形移位器。

2、子模块描述（本实验只有主电路一个模块）

原理图（借用 8 位桶形移位器，本题与其类似）

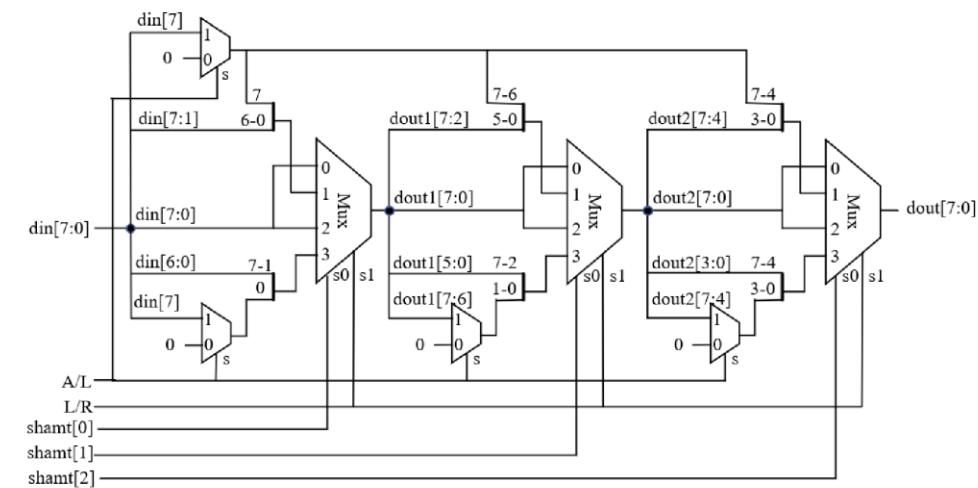
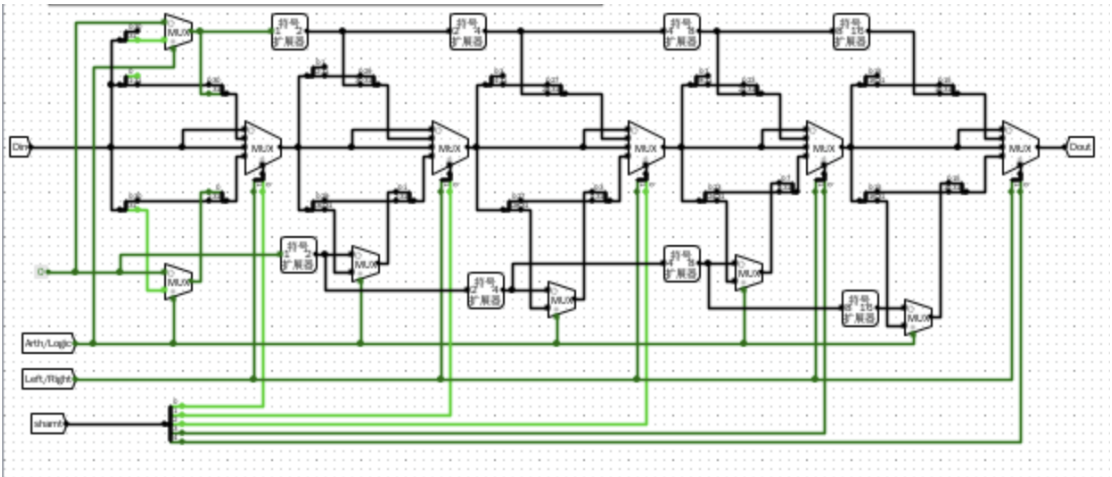


图 2-13 8 位桶形移位器原理图

电路图：



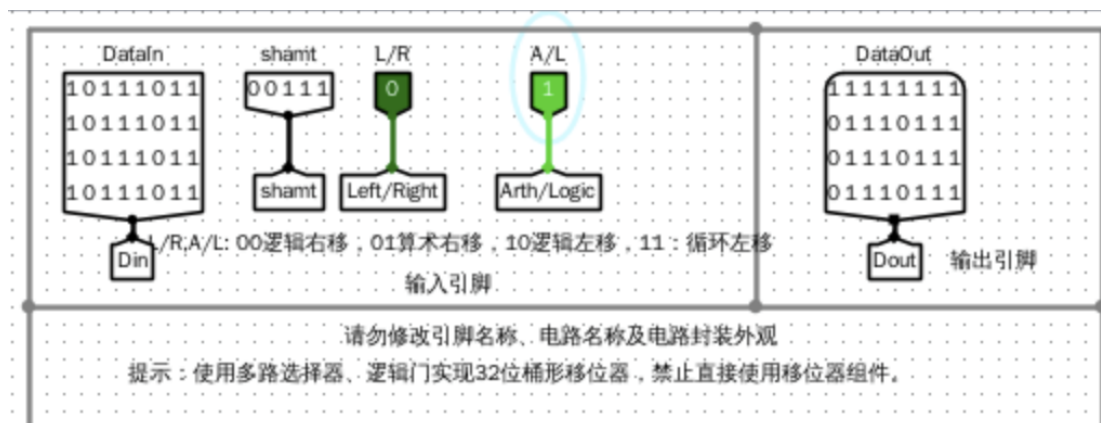
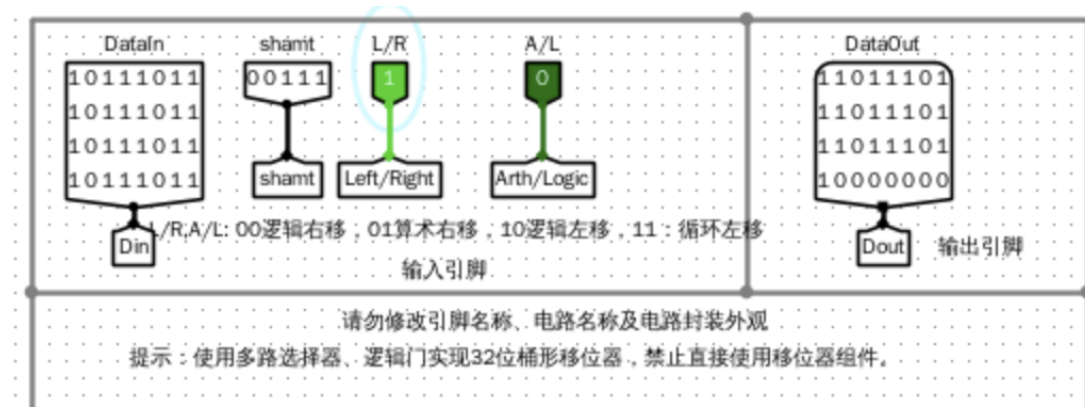
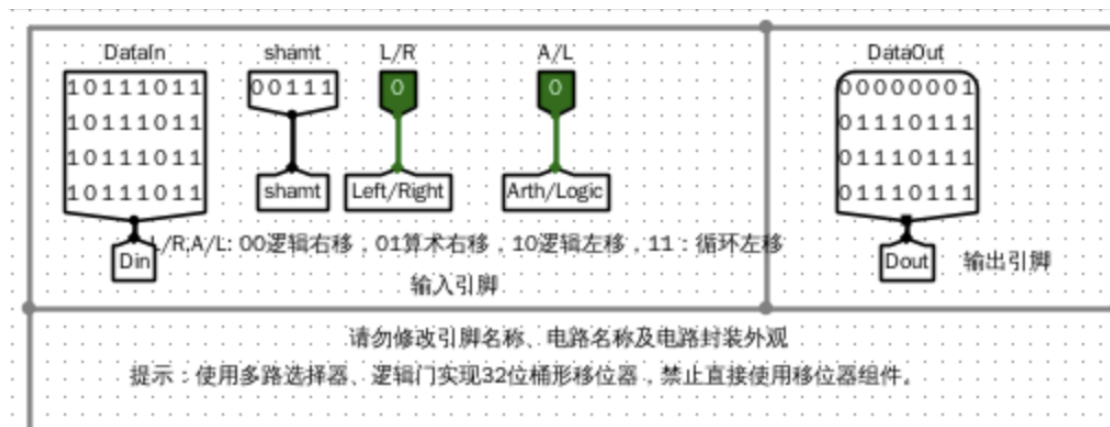
接口功能：

接口方向	接口名称	位宽	作用说明
输入	Din[31:0]	32 位	待移位的 32 位输入数据
输入	shamt[4:0]	5 位	移位位数 (0~31 位)
输入	L/R	1 位	移位方向: 0 = 右移, 1 = 左移
输入	A/L	1 位	移位类型: 0 = 逻辑移位, 1 = 算术移位
输出	Dout[31:0]	32 位	移位完成后的输出结果

3.功能表（列举部分移位情况）

Cnt	Din	shamt	LR	AL	Dout
00	ffffffff	01	0	0	7ffffffff
01	ffffffff	01	1	0	ffffffffe
02	ffffffff	01	0	1	ffffffff
03	ffffffff	01	1	1	ffffffff
04	bbbbbbbb	07	0	0	01777777
05	bbbbbbbb	07	1	0	dddddd80
06	bbbbbbbb	07	0	1	ff777777
07	bbbbbbbb	07	1	1	dddddddd

4. 仿真模拟：



5. 错误现象及分析：

一开始是用位扩展器的时候选择了错误的输入扩展方式，改为符号扩展后解决了问题。

五、32 位 ALU 设计：

1.整体模块设计：利用之前设计的 32 位 ALU，32 位 Shifter，以及逻辑门阵列完成，其核心步骤为将 4 位的 ALUctr 信号转换为 SUBctr（减法信号），SIGctr（带/无符号整数比较信号），LRctr（移位方向信号），ALctr（移位方式信号）和 OPctr（控制最终输出什么运算的结果）。其中 ALUctr 编码对应的各个信号的值如下表：

A	B	C	D	E	F	G	H	I
ALUctr<3:0>	指令操作类型	SUBctr	SIGctr	ALctr	LRctr	OPctr<2:0>	OPctr 含义	
0000	auipc,addi,add,jal,jalr,lb,lh,lw,lbu,lhu,sb,sh,sw: 加法	0	x	x	x	000	选择加法器结果输出	
0001	sll,slli: 逻辑左移	x	x	0	1	100	选择移位器结果输出	
0010	slt,slti,beq,bne,blt,bge: 带符号数小于比较	1	1	x	x	110	选择小于置位结果输出	
0011	sltu,sltui,bltu,bgeu: 无符号数小于比较	1	0	x	x	110	选择小于置位结果输出	
0100	xor,xori: 异或	x	x	x	x	011	选择按位异或结果输出	
0101	srl,srli: 逻辑右移	x	x	0	0	100	选择移位器结果输出	
0110	or,ori: 或运算	x	x	x	x	010	选择按位或结果输出	
0111	and,andi: 与运算	x	x	x	x	001	选择按位与结果输出	
1000	sub: 减法	1	x	x	x	000	选择加法器结果输出	
1101	sra,srai: 算术右移	x	x	1	0	100	选择移位器结果输出	
1111	lui: 取操作数 B	x	x	x	x	101	选择操作数 B 直接输出	

按照最小风险法进行化简。控制信号的最小项表达式如下：

SUBctr = $\sum m(2,3,8)$

SIGctr = m_2

ALctr = m_{13}

LRctr = m_1

Opctr[2]= $\sum m(1,2,3,5,13,15)$

Opctr[1]= $\sum m(2,3,4,6)$

Opctr[0]= $\sum m(4,7, 15)$

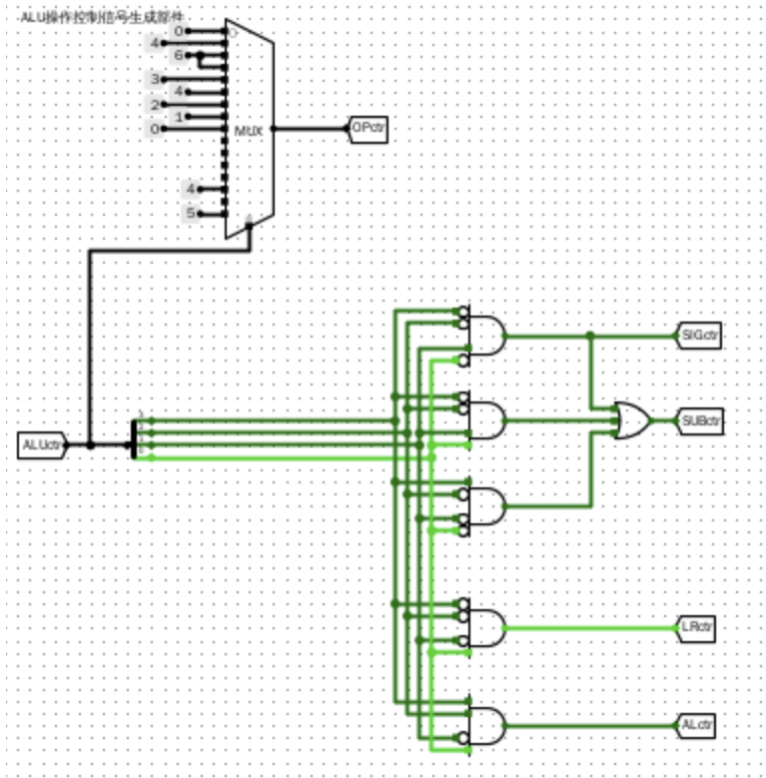
（实际设计中，OPctr 信号是用比较器完成，其余是用与门与或门阵列完成）利用设计好的 ALUctr 部件配合直接完成的移位器，加法器与逻辑门阵列，多路选择器就完成了 32 位 ALU 的设计。

2.子模块原理图与描述：

(在此仅描述 ALUctr 与 ALU 部分，其余与上面的设计相同，不再赘述)

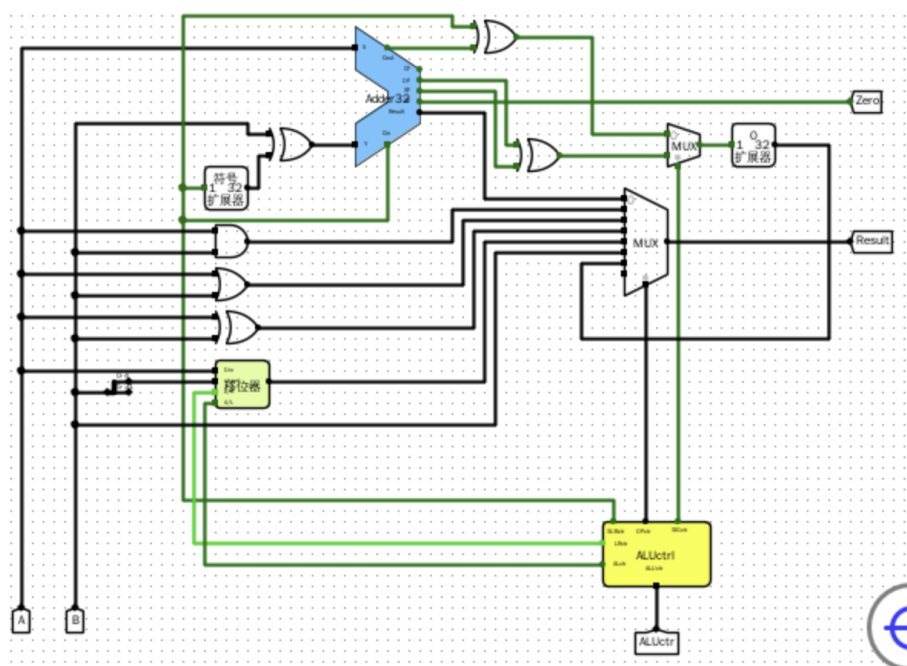
1) ALUctr：原理在整体模块设计中已说明完毕。

电路图：



接口功能：

接口方向	接口名称	位宽	作用说明
输入	ALUctr[3:0]	4 位	ALU 操作控制编码
输出	SUBctr	1 位	加 / 减控制：0 = 加，1 = 减
输出	SIGctr	1 位	比较类型：0 = 无符号，1 = 有符号



接口功能：

接口方向	接口名称	位宽	作用说明
输入	A[31:0]	32 位	第一个 32 位操作数
输入	B[31:0]	32 位	第二个 32 位操作数
输入	ALUctr[3:0]	4 位	ALU 运算功能选择
输出	Result[31:0]	32 位	ALU 最终运算结果
输出	Zero	1 位	零标志：结果全 0 时为 1

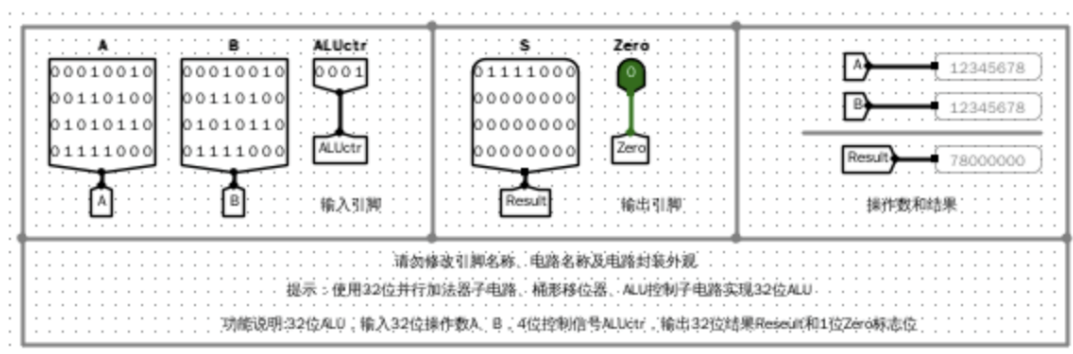
3.功能表

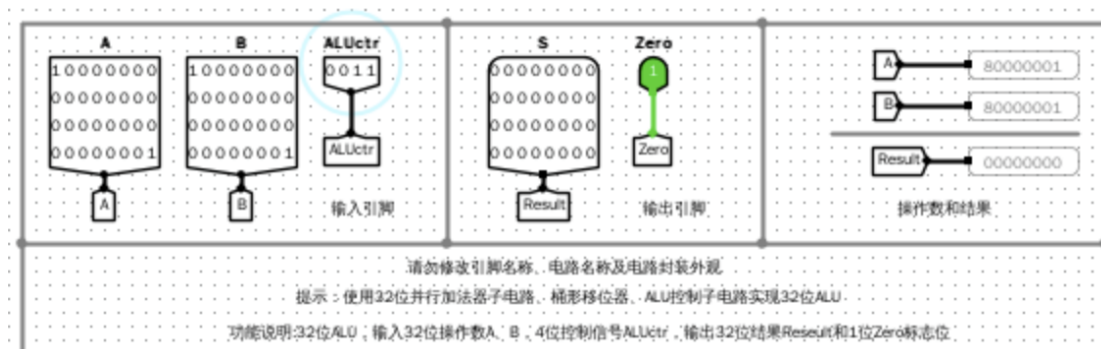
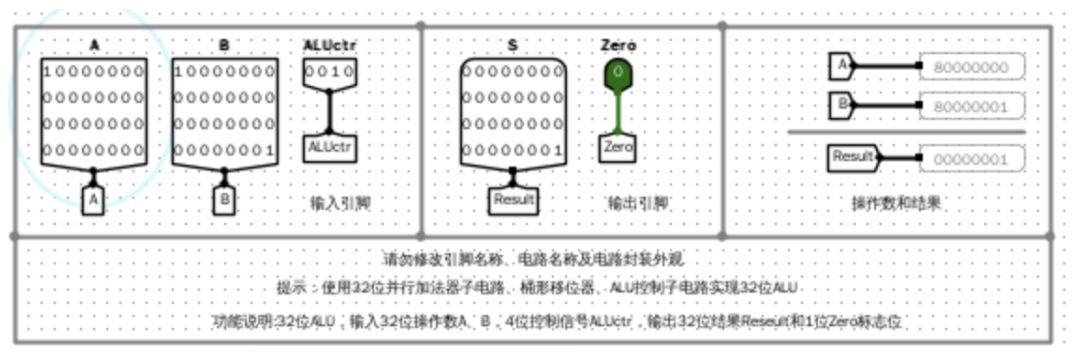
序号	A	B	ALUctr<3:0>	Result	Zero
1	0x80000000	0x7fffffff	0000	0xffffffff	0
2	0x01	0x7fffffff	0001	0x80000000	0
3	0x80000000	0x7fffffff	0010	0x00000001	0
4	0x80000000	0x7fffffff	0011	0x00000000	0
5	0x80000000	0x7fffffff	0100	0xffffffff	0
6	0x80000000	0x7fffffff	0101	0x00000001	0
7	0x80000000	0x7fffffff	0110	0xffffffff	0
8	0x80000000	0x7fffffff	0111	0x00000000	0
9	0x80000000	0x7fffffff	1000	0x00000001	0
10	0x80000000	0x7fffffff	1101	0xffffffff	0
11	0x80000000	0x7fffffff	1111	0x7fffffff	0
12	0x80000000	0x80000000	0000	0x00000000	1

序号	A	B	ALUctr<3:0>	Result	Zero
13	0x80000000	0x80000000	0010	0x00000000	1
14	0x80000000	0x80000000	0011	0x00000000	1
15	0x80000000	0x80000000	0100	0x00000000	1
16	0x80000000	0x80000000	0101	0x80000000	1
17	0x80000000	0x80000000	0110	0x80000000	1
18	0x80000000	0x80000000	0111	0x80000000	1
19	0x80000000	0x80000000	1000	0x00000000	1
20	0x80000000	0x80000000	1101	0x80000000	1
21	0x80000000	0x80000000	1111	0x80000000	1
22	0x5a5a5a5a	0xa5a5a5a5	0000	0xffffffff	0
23	0x5a5a5a5a	0xa5a5a5a5	0001	0x4b4b4b40	0
24	0x5a5a5a5a	0xa5a5a5a5	0010	0x00000000	0

序号	A	B	ALUctr<3:0>	Result	Zero
25	0x5a5a5a5a	0xa5a5a5a5	0011	0x00000001	0
26	0x5a5a5a5a	0xa5a5a5a5	0100	0xffffffff	0
27	0x5a5a5a5a	0xa5a5a5a5	0101	0x02d2d2d2	0
28	0x5a5a5a5a	0xa5a5a5a5	0110	0xffffffff	0
29	0x5a5a5a5a	0xa5a5a5a5	0111	0x00000000	0
30	0x5a5a5a5a	0xa5a5a5a5	1000	0xb4b4b4b5	0
31	0x5a5a5a5a	0xa5a5a5a5	1101	0x02d2d2d2	0
32	0x5a5a5a5a	0xa5a5a5a5	1111	0xa5a5a5a5	0

4.仿真模拟：





5. 错误现象及分析：

一开始是无法通过 $A=12345678$ ， $B=12345678$ ， $ALUctr=1$ 的测试（逻辑左移），结果 78000000 正确，但是 ZF 位显示为 1。经过检查发现 ALUctr 中 SUBctr 信号连接错误导致这里错误运行了减法运算使 ZF 为 1。更正错误后结果正确。

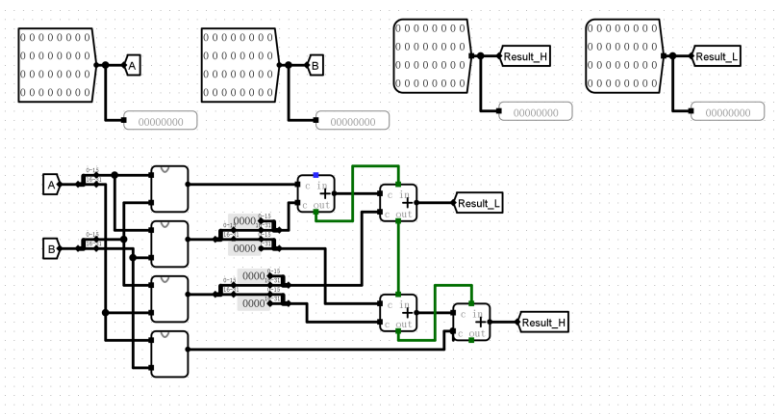
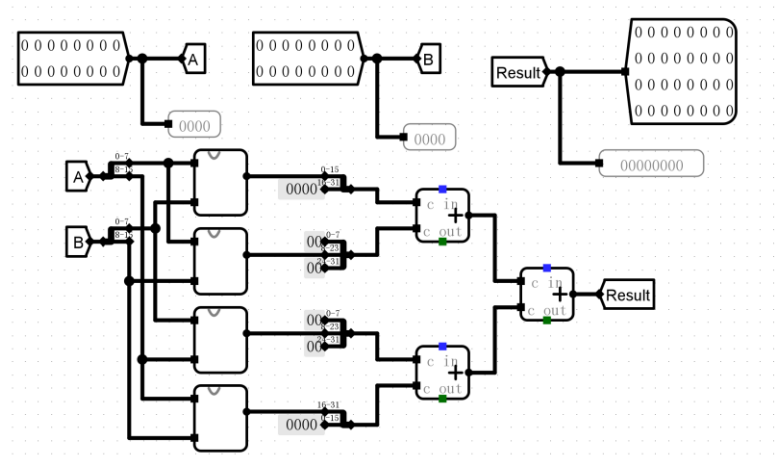
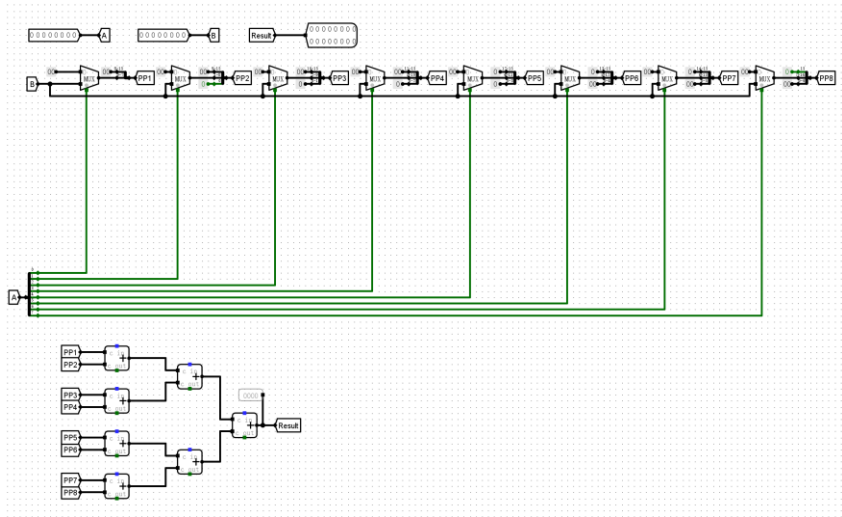
六、思考题：

1. 设计 64 位快速加法器：使用 4 个 16 位并行进位加法器与 CLU4，组间快速计算进位，实现分层并行进位加法器（类似于 16 位 CLA 时使用 4 个 4 位 CLA 和 CLU4）

2. 32 位快速乘法器：

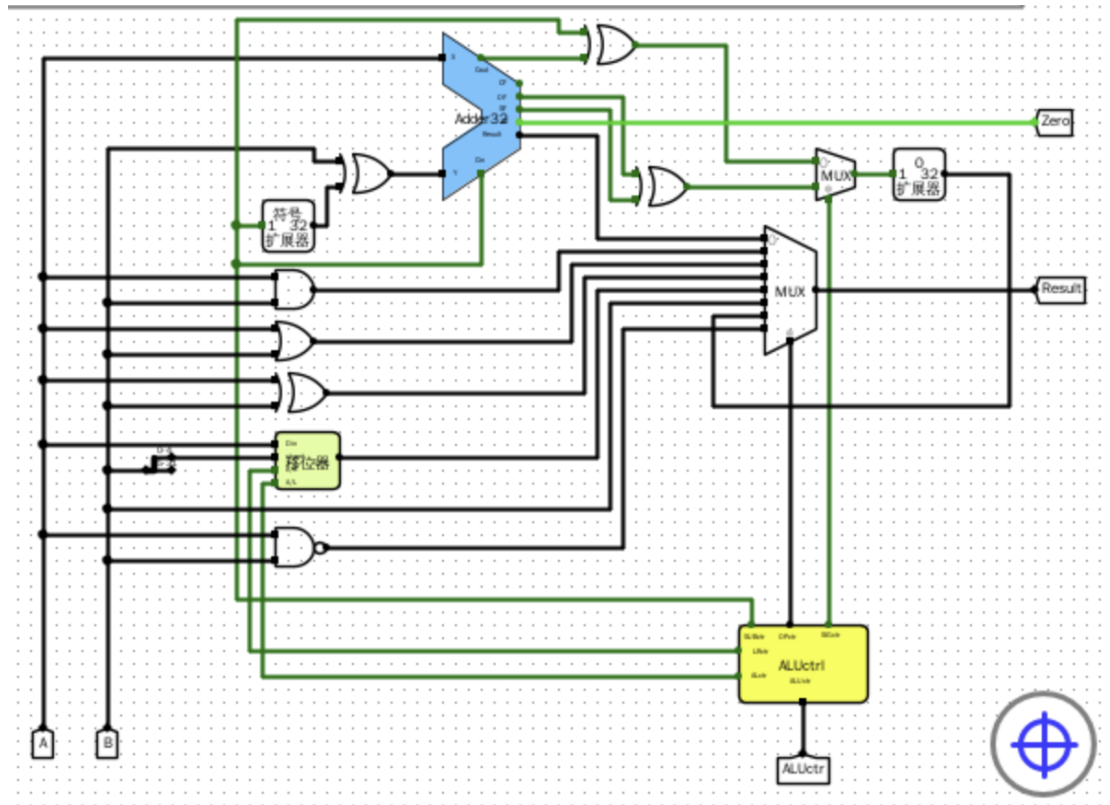
先制作 8 位快速乘法器：用多路选择器获得 8 个部分积，用适当的 0 位

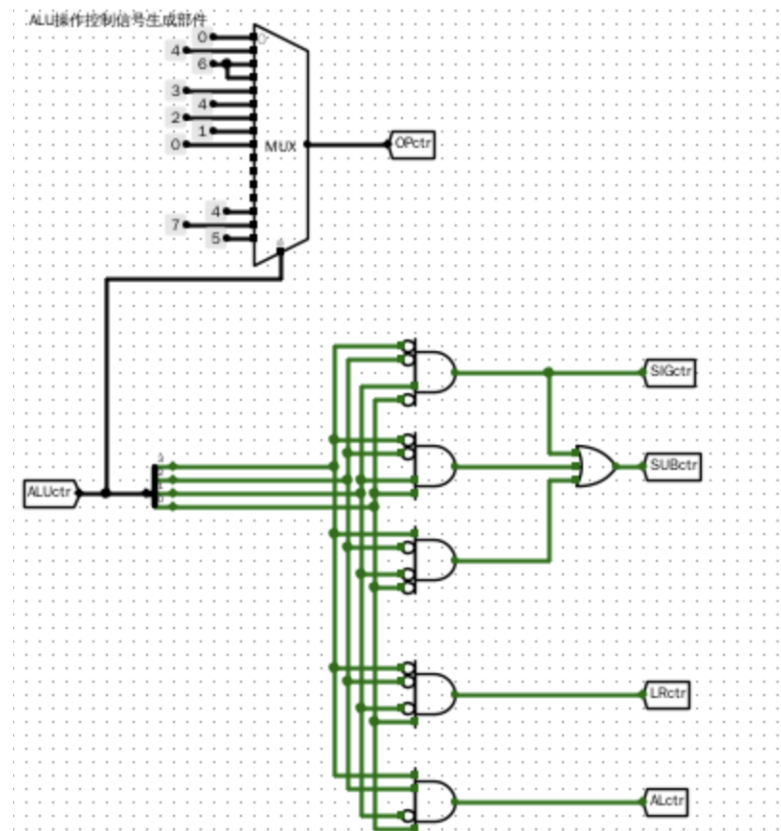
扩展之后使用加法器得到结果。再扩展为 16 位快速乘法器，最后制成 32 位快速乘法器。



3.ALU 新增一个指令：

新增按位与非指令，选取 ALUctr 原来未用态 1110，对应的 SUBctr, SIGctr, ALctr 和 LRctr 均为 0，OPctr 为 111。





4. 分析比较运算使用独立的比较器和使用减法运算通过标志位来实现两种方法的特性：

独立比较器的优势在于速度快、面积小、功耗低，适合需要频繁比较的高性能设计。

减法+标志位的优势在于不需要额外硬件，功能更灵活（一票多用），适合面积受限的嵌入式设计。

现代高性能 CPU 通常同时实现两者：比较指令走独立快速路径，算术指令的减法仍产生标志位供分支指令使用。